// Security Assessment

10.24.2025 - 10.29.2025

Alcum Smart Contracts

Coper Investment

HALBORN

Alcum Smart Contracts - Coper Investment

Prepared by: HALBORN

Last Updated 11/19/2025

Date of Engagement: October 24th, 2025 - October 29th, 2025

Summary

100% ⓘ OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

ALL FINDINGS	CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL
3	0	0	1	2	0

TABLE OF CONTENTS

1. Introduction

`Coper Investment` engaged `Halborn` to perform a security assessment of their `Alcum` smart contracts beginning on October 24th, 2025 and ending on October 29th, 2025. The assessment scope was limited to the smart contracts provided to Halborn. Commit hashes and additional details are available in the Scope section of this report.

`Alcum` is a DeFi protocol that allows users to invest in copper-backed assets by depositing various tokens, which are converted to copper-backed CUP tokens held in an investment vault with automated profit distribution.

2. Assessment Summary

`Halborn` assigned `1` full-time security engineer to conduct a comprehensive review of the smart contracts within scope. The engineer is an expert in blockchain and smart contract security, with advanced skills in penetration testing and smart contract exploitation, as well as extensive knowledge of multiple blockchain protocols.

The objectives of this assessment were to:

- Identify potential security vulnerabilities within the smart contracts.
- Verify that the smart contract functionality operates as intended.

In summary, `Halborn` identified several areas for improvement to reduce the likelihood and impact of security risks, which were completely addressed by the `Coper Investment team`. The main ones were:

- `Add slippage protection to all token swaps`
- `Validate that users cannot supply both ETH and tokens simultaneously`
- `Use system-generated unique identifiers instead of user-supplied ones for deposit IDs`

3. Test Approach And Methodology

`Halborn` performed a combination of a manual review of the source code and automated security testing to balance efficiency, timeliness, practicality, and accuracy in regard to the scope of the program assessment. While manual testing is recommended to uncover flaws in business logic, processes, and implementation; automated testing techniques help enhance coverage of programs and can quickly identify items that do not follow security best practices.

The following phases and associated tools were used throughout the term of the assessment:

- Research into the architecture and purpose of the smart contracts.
- Manual code review and walkthrough of the smart contracts.
- Manual assessment of critical Solidity variables and functions to identify potential vulnerability classes.
- Manual testing using custom scripts.
- Static security analysis of the scoped contracts and imported functions.
- Local deployment and testing with Foundry & Hardhat.

4. RISK METHODOLOGY

Every vulnerability and issue observed by Halborn is ranked based on **two sets** of **Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

4.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker's control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A) Specific (AO:S)	1 0.2
Attack Cost (AC)	Low (AC:L) Medium (AC:M) High (AC:H)	1 0.67 0.33

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Complexity (AX)	Low (AX:L) Medium (AX:M) High (AX:H)	1 0.67 0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

4.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

IMPACT METRIC (M_I)	METRIC VALUE	NUMERICAL VALUE
Confidentiality (C)	None (C:N) Low (C:L) Medium (C:M) High (C:H) Critical (C:C)	0 0.25 0.5 0.75 1

IMPACT METRIC (M_I)	METRIC VALUE	NUMERICAL VALUE
Integrity (I)	None (I:N) Low (I:L) Medium (I:M) High (I:H) Critical (I:C)	0 0.25 0.5 0.75 1
Availability (A)	None (A:N) Low (A:L) Medium (A:M) High (A:H) Critical (A:C)	0 0.25 0.5 0.75 1
Deposit (D)	None (D:N) Low (D:L) Medium (D:M) High (D:H) Critical (D:C)	0 0.25 0.5 0.75 1
Yield (Y)	None (Y:N) Low (Y:L) Medium (Y:M) High (Y:H) Critical (Y:C)	0 0.25 0.5 0.75 1

Impact I is calculated using the following formula:

$$I = max(m_I) + \frac{\sum m_I - max(m_I)}{4}$$

4.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

SEVERITY COEFFICIENT (C)	COEFFICIENT VALUE	NUMERICAL VALUE
Reversibility (r)	None (R:N) Partial (R:P) Full (R:F)	1 0.5 0.25
Scope (s)	Changed (S:C) Unchanged (S:U)	1.25 1

Severity Coefficient C is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score S is obtained by:

$$S = min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

SEVERITY	SCORE VALUE RANGE
Critical	9 - 10
High	7 - 8.9
Medium	4.5 - 6.9
Low	2 - 4.4
Informational	0 - 1.9

5. SCOPE

REPOSITORY

(a) Repository: alcum-smart

(b) Assessed Commit ID: 642a824

(c) Items in scope:

- libraries/RedeemLib.sol
- CopperPriceConsumer.sol
- CUPToken.sol
- EpochManager.sol
- HostAdapter.sol
- SettlementEngine.sol
- Silo.sol
- xCup.sol
- Zapper.sol

Out-of-Scope: Third party dependencies and economic attacks.

REMEDIATION COMMIT ID:

- f45d22a

Out-of-Scope: New features/implementations after the remediation commit IDs.

6. ASSESSMENT SUMMARY & FINDINGS OVERVIEW

CRITICAL	HIGH	MEDIUM	LOW
0	0	1	2

INFORMATIONAL
0

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
SUPPLYING BOTH ETH AND ERC20 TOKEN DURING DEPOSIT WILL LEAD TO LOSS OF ETH	MEDIUM	SOLVED - 11/10/2025
FRAGILE DEPOSIT ID GENERATION AND HANDLING	LOW	SOLVED - 11/10/2025
MISSING SLIPPAGE PROTECTION ON ETH SWAPS	LOW	SOLVED - 11/10/2025

7. FINDINGS & TECH DETAILS

7.1 SUPPLYING BOTH ETH AND ERC20 TOKEN DURING DEPOSIT WILL LEAD TO LOSS OF ETH

// MEDIUM

Description

The `zapAndDeposit()` and `zapAndDepositWithPermit()` functions are marked as payable, allowing users to send ETH, but they also accept ERC20 tokens as input. The contract fails to validate that only one asset type is supplied per transaction. If a user sends both ETH (`msg.value`) and an ERC20 token, the function processes the ERC20 deposit but silently keeps the ETH in the contract. Since the ETH is never refunded or retrievable, it becomes permanently stuck.

Code Location

The `Zapper::zapAndDepositWithPermit` processes only the ERC20 token supplied, ignoring the ETH (if simultaneously supplied):

```
564 function zapAndDepositWithPermit(
565     IERC20 tokenIn,
566     uint256 amount,
567     PermitParams calldata permitParams,
568     bytes32 depositId,
569     uint256 slippageBps
570 ) external payable whenNotPaused {
571     // allow address(0) for ETH
572     require(amount != 0, "Invalid amount");
573 
574     // Use default slippage of 1% (100 basis points) if 0 is passed
575     uint256 actualSlippage = slippageBps == 0 ? 100 : slippageBps;
576 
577     if (address(tokenIn) != address(0)) {
578         if (tokenIn.allowance(_msgSender(), address(this)) < amount) {
579             _execPermit(tokenIn, _msgSender(), address(this), permitParams);
580         }
581     } else {
582         // ETH: require msg.value
583         require(msg.value == amount, "Invalid ETH amount");
584     }
585 
586     uint256 depositValue = _processDeposit(tokenIn, amount, actualSlippage);
587     _recordDeposit(depositId, depositValue);
588 
589     emit ZapAndDeposit(router(), address(tokenIn), depositValue);
590 }
```

Same as the `Zapper::zapAndDeposit` function:

```
603 function zapAndDeposit(IERC20 tokenIn, uint256 amount, bytes32 depositId, uint256 slippageBps)
604     external
605     payable
606     whenNotPaused
607 {
608     // allow address(0) for ETH
609     require(amount != 0, "Invalid amount");
610 
611     // Use default slippage of 1% (100 basis points) if 0 is passed
612     uint256 actualSlippage = slippageBps == 0 ? 100 : slippageBps;
613 
614     if (address(tokenIn) == address(0)) {
```

```
615             require(msg.value == amount, "Invalid ETH amount");
616         }
617
618         uint256 depositValue = _processDeposit(tokenIn, amount, actualSlippage);
619         _recordDeposit(depositId, depositValue);
620
621         emit ZapAndDeposit(router(), address(tokenIn), depositValue);
622     }
```

`Zapper::_processDeposit` function:

```
540 function _processDeposit(IERC20 tokenIn, uint256 amount, uint256 slippageBps)
541     internal
542     returns (uint256 depositValue)
543 {
544     if (address(tokenIn) == address(_usdc)) {
545         tokenIn.safeTransferFrom(_msgSender(), address(_silo), amount);
546         depositValue = amount;
547     } else {
548         depositValue = _zapIn(tokenIn, amount, slippageBps);
549     }
550 }
```

Proof of Concept

Users can accidentally lose ETH if they send both ETH and an ERC20 token to the deposit function. The contract processes the token but ignores the ETH, leaving it permanently stuck with no recovery option.

Test Setup

1. Deploy mock contracts: CUP token, xCUP vault, USDC token, Uniswap router, and price oracle
2. Initialize all contracts with proper roles and permissions
3. Mint 10,000 USDC to the test account

Exploit Steps

1. Allocate 1 ETH to send as `msg.value`.
2. Allocate 1000 USDC to send as ERC20 token parameter.
3. Create a unique deposit ID.
4. Capture the Zapper contract's ETH balance before the transaction.
5. Call `zapAndDeposit(USDC_ADDRESS, 1000e6, depositId, 100)` with `msg.value = 1 ETH`.
6. This sends both ETH and USDC to the contract simultaneously.
7. Capture the Zapper contract's ETH balance after the transaction.
8. Assert that the contract balance increased.
9. Assert that the increase equals exactly 1 ETH.
10. This proves the ETH was accepted but never processed.

```
pragma solidity ^0.8.24;
import {Test} from "forge-std/Test.sol";
import {Zapper} from "../contracts/Zapper.sol";
import {CUPToken} from "../contracts/CUPToken.sol";
import {xCUP} from "../contracts/xCUP.sol";
import {EpochManager} from "../contracts/EpochManager.sol";
import {HostAdapter} from "../contracts/HostAdapter.sol";
import {ERC1967Proxy} from "@openzeppelin/contracts/proxy/ERC1967/ERC1967Proxy.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {ICopperPriceConsumer} from "../contracts/interfaces/ICopperPriceConsumer.sol";
contract MockCopperPriceConsumer is ICopperPriceConsumer {
```

```
    uint256 public price = 450000000;
    function requestCopperPrice() external pure returns (bytes32) {
        return bytes32(0);
    }
    function fulfill(bytes32, uint256) external pure {
        revert("Not implemented");
    }
    function getPriceAsDecimal() external view returns (uint256) {
        return price / 10 ** 8;
    }
    function updatePrice(uint256 _price) external {
        price = _price;
    }
}
contract MockUSDC is IERC20 {
    mapping(address => uint256) public balanceOf;
    mapping(address => mapping(address => uint256)) public allowance;
    function decimals() external pure returns (uint8) {
        return 6;
    }
    function totalSupply() external pure returns (uint256) {
        return 0;
    }
    function mint(address to, uint256 amount) external {
        balanceOf[to] += amount;
    }
    function transfer(address to, uint256 amount) external returns (bool) {
        balanceOf[msg.sender] -= amount;
        balanceOf[to] += amount;
        return true;
    }
    function approve(address spender, uint256 amount) external returns (bool) {
        allowance[msg.sender][spender] = amount;
        return true;
    }
    function transferFrom(address from, address to, uint256 amount) external returns (bool) {
        require(balanceOf[from] >= amount, "Insufficient balance");
        require(allowance[from][msg.sender] >= amount, "Insufficient allowance");
        balanceOf[from] -= amount;
        balanceOf[to] += amount;
        allowance[from][msg.sender] -= amount;
        return true;
    }
}
contract MockUniswapRouter {
    function WETH() external pure returns (address) {
        return address(0x1234567890123456789012345678901234567890);
    }
    function getAmountsOut(uint256 amountIn, address[] calldata) external pure returns (uint256[] mem
        amounts = new uint256[](2);
        amounts[0] = amountIn;
        amounts[1] = amountIn;
    }
    function swapExactETHForTokens(uint256, address[] calldata, address, uint256)
        external
        payable
        returns (uint256[] memory amounts)
    {
        amounts = new uint256[](2);
        amounts[0] = msg.value;
        amounts[1] = msg.value;
    }
    function swapExactTokensForTokens(uint256 amountIn, uint256, address[] calldata, address, uint256
        external
        pure
        returns (uint256[] memory amounts)
    {
        amounts = new uint256[](2);
        amounts[0] = amountIn;
        amounts[1] = amountIn;
    }
}
contract FIND002Test is Test {
    Zapper zapper;
    MockUSDC usdc;
    CUPToken cup;
    xCUP xcup;
    EpochManager epochManager;
```

```
    MockCopperPriceConsumer priceConsumer;
    MockUniswapRouter router;
    function setUp() public {
        priceConsumer = new MockCopperPriceConsumer();
        usdc = new MockUSDC();
        router = new MockUniswapRouter();
        CUPToken cupImpl = new CUPToken();
        bytes memory cupInitData = abi.encodeWithSelector(CUPToken.initialize.selector);
        ERC1967Proxy cupProxy = new ERC1967Proxy(address(cupImpl), cupInitData);
        cup = CUPToken(address(cupProxy));
        EpochManager epochImpl = new EpochManager();
        bytes memory epochInitData = abi.encodeWithSelector(EpochManager.initialize.selector, 7 days)
        ERC1967Proxy epochProxy = new ERC1967Proxy(address(epochImpl), epochInitData);
        epochManager = EpochManager(address(epochProxy));
        xCUP xcupImpl = new xCUP();
        bytes memory xcupInitData = abi.encodeWithSelector(
            xCUP.initialize.selector,
            IERC20(address(cup)),
            "xCUP Vault",
            "xCUP",
            address(priceConsumer),
            address(router),
            address(usdc),
            address(0x1234567890123456789012345678901234567890)
        );
        ERC1967Proxy xcupProxy = new ERC1967Proxy(address(xcupImpl), xcupInitData);
        xcup = xCUP(address(xcupProxy));
        Zapper zapperImpl = new Zapper();
        bytes memory zapperInitData = abi.encodeWithSelector(
            Zapper.initialize.selector,
            address(cup),
            address(usdc),
            address(xcup),
            address(router),
            address(priceConsumer),
            address(epochManager)
        );
        ERC1967Proxy zapperProxy = new ERC1967Proxy(address(zapperImpl), zapperInitData);
        zapper = Zapper(payable(address(zapperProxy)));
        cup.grantRole(keccak256("MINTER_ROLE"), address(zapper));
        usdc.mint(address(this), 10000e6);
        usdc.approve(address(zapper), type(uint256).max);
    }
    function testETHLossWhenBothETHAndERC20Supplied() public {
        uint256 usdcAmount = 1000e6;
        uint256 ethAmount = 1 ether;
        bytes32 depositId = keccak256(abi.encodePacked("test-deposit"));
        uint256 zapperBalanceBefore = address(zapper).balance;
        zapper.zapAndDeposit{value: ethAmount}(IERC20(address(usdc)), usdcAmount, depositId, 100);
        uint256 zapperBalanceAfter = address(zapper).balance;
        assert(zapperBalanceAfter > zapperBalanceBefore);
        assert(zapperBalanceAfter - zapperBalanceBefore == ethAmount);
    }
}
```

The PoC passes and demonstrates that it was possible to send 1 ETH to the deposit function along with 1000 USDC, and the ETH remains permanently stuck in the contract with no recovery mechanism.

```
[PASS] testETHLossWhenBothETHAndERC20Supplied() (gas: 259885)
Suite result: ok. 1 passed; 0 failed; 0 skipped
```

BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:N/D:H/Y:N (5.0)

Recommendation

Add validation checks to ensure mutual exclusivity between ETH and ERC20 deposits in both `zapAndDeposit()` and `zapAndDepositWithPermit()` functions. This guarantees that only one deposit type is accepted, eliminating the possibility of user fund loss.

Remediation Comment

SOLVED: The **Alcum team** resolved the issue by adding validation checks to the deposit functions to require `msg.value == 0` when depositing ERC20 tokens, preventing accidental ETH loss.

Remediation Hash

https://github.com/unusnullus/alcum-smart/commit/f45d22a8d595447a7aa410e1eea03d5088d5c01e

7.2 FRAGILE DEPOSIT ID GENERATION AND HANDLING

// LOW

Description

The **Zapper** contract uses weak deposit ID generation schemes, introducing two potential issues that could affect data integrity and user operations:

1. **Front-running Denial of Service (DoS):** The `_recordDeposit()` function relies on user-supplied `depositId` values. Since these IDs are predictable, an attacker can front-run a user's deposit using the same ID, permanently blocking the legitimate deposit from succeeding. This allows any malicious actor to repeatedly prevent deposits from being recorded.
2. **Data Overwrite from Hash Collisions:** The `_recordExternalDeposit()` function generates IDs with `keccak256(abi.encodePacked(..., block.timestamp, ...))`. Multiple deposits with identical parameters within the same block can generate identical IDs. Without a collision check, this causes existing deposits to be silently overwritten, erasing prior data and potentially making approved deposits unclaimable.

These can result in failed deposits, corrupted records, or the permanent loss of user balances and approval data.

Code Location

Fragile depositId handling in the `Zapper::_recordDeposit` function:

```
372 function _recordDeposit(bytes32 depositId, uint256 amount) internal {
373     require(_approvedDeposits[depositId].user == address(0), "Deposit ID already exists");
374
375     _approvedDeposits[depositId] = Deposit({
376         user: _msgSender(),
377         depositId: depositId,
378         amount: amount,
379         approvedAmount: 0,
380         approved: false,
381         beneficiary: address(0),
382         createdBy: _msgSender(),
383         claimedBy: address(0),
384         isExternal: false,
385         tag: bytes32(0),
386         approvedCupAmount: 0,
387         priceSnapshot: 0
388     });
389     _pendingDepositIds.push(depositId);
390     _userDeposits[_msgSender()].push(depositId);
391 }
```

As well as in the `Zapper::_recordExternalDeposit` function:

```
404 function _recordExternalDeposit(uint256 usdcAmount, address beneficiary_, bytes32 tag_)
405     internal
406     returns (bytes32 depositId)
407 {
408     require(beneficiary_ != address(0), "Invalid beneficiary");
409     depositId = keccak256(abi.encodePacked(_msgSender(), block.timestamp, usdcAmount, benefici
410     _approvedDeposits[depositId] = Deposit({
411         user: _msgSender(),
412         depositId: depositId,
413         amount: usdcAmount,
```

```
            approvedAmount: 0,
            approved: false,
            beneficiary: beneficiary_,
            createdBy: _msgSender(),
            claimedBy: address(0),
            isExternal: true,
            tag: tag_,
            approvedCupAmount: 0,
            priceSnapshot: 0
        });
        _pendingDepositIds.push(depositId);
        _userDeposits[beneficiary_].push(depositId);
    }
```

BVSS

AO:A/AC:M/AX:L/R:N/S:U/C:N/A:N/I:M/D:N/Y:N (3.4)

Recommendation

Consider using a nonce-based approach to ensure each deposit is uniquely linked to the depositor.

Remediation Comment

SOLVED: The **Alcum team** resolved the issue by replacing user-supplied deposit IDs with nonce-based generation that includes collision detection and automatic retries.

Remediation Hash

https://github.com/unusnullus/alcum-smart/commit/f45d22a8d595447a7aa410e1eea03d5088d5c01e

7.3 MISSING SLIPPAGE PROTECTION ON ETH SWAPS

// LOW

Description

The `Zapper` contract's ETH swap function lacks slippage protection. When performing swaps on Uniswap V2, it passes the expected output as the minimum acceptable output instead of applying a slippage-adjusted value. This causes transactions to revert whenever token prices change slightly during execution, which is a common occurrence due to normal market movement or MEV activity.

Because ETH swaps demand exact output amounts, any minor deviation leads to failed transactions and wasted gas.

Code Location

Absence of slippage inclusion in `Zapper::_tradeForToken` function during ETH swaps:

```
476 function _tradeForToken(address tokenIn, address tokenOut, uint256 amountIn, uint256 slippageB
477     // Construct trading path for Uniswap V2 (direct pair assumed)
478     address[] memory path = new address[](2);
479
480     // Handle ETH representation in trading path
481     if (tokenIn == address(0)) {
482         // ETH case: Use WETH address for Uniswap routing
483         path[0] = _router.WETH();
484     } else {
485         // ERC20 token case: Use token address directly
486         path[0] = tokenIn;
487     }
488     path[1] = tokenOut;
489
490     // Query expected output amounts from Uniswap router
491     uint256[] memory amountsOut = _router.getAmountsOut(amountIn, path);
492
493     // Handle ETH swaps specially (require ETH to be sent with transaction)
494     if (tokenIn == address(0) || tokenIn == _router.WETH()) {
495         // send ETH along
496         _router.swapExactETHForTokens{value: amountIn}(amountsOut[1], path, address(_silo), bl
497         return;
498     }
499
500     // Calculate minimum output with custom slippage tolerance
501     uint256 minOutput = (amountsOut[1] * (10000 - slippageBps)) / 10000;
502
503     _router.swapExactTokensForTokens(amountIn, minOutput, path, address(_silo), block.timestam
504 }
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N (2.5)

Recommendation

Add slippage tolerance to the ETH swap logic, similar to ERC20 swaps, this ensures transactions tolerate small price changes, preventing unnecessary reverts.

Remediation Comment

SOLVED: The **Alcum team** resolved the issue by implementing proper slippage calculation that applies to all token swaps, including ETH.

Remediation Hash

https://github.com/unusnullus/alcum-smart/commit/f45d22a8d595447a7aa410e1eea03d5088d5c01e

---

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.